Estructuras de Datos

ICPC camp Potosí 2026

Estructuras de Datos

Definición:

Las estructuras de datos son formas de organizar y almacenar datos de manera que podamos realizar consultas (queries) y modificaciones de manera eficiente.

Propósito:

- Facilitar la respuesta a consultas sobre un conjunto de datos.
- Permitir modificaciones en los datos durante el proceso de consulta.

Importancia en Programación Competitiva:

- Optimización de algoritmos.
- Resolución eficiente de problemas complejos.

Operador Asociativo

Definición:

Un operador binario $\oplus$ se dice asociativo, si siempre se cumple que $(a \oplus b) \oplus c = a \oplus (b \oplus c)$

Ejemplos:

- Suma + de enteros (o reales)
- Producto * de enteros (o reales)
- Máximo / Mínimo de enteros (o reales)
- Operadores lógicos “or”, “and” y “xor”

Operador Asociativo

El operador podría no ser conmutativo:

- Producto de matrices
- Composición de funciones
- Concatenación de Strings

El operador podría no tener inverso:

- Máximo / Mínimo
- Operadores lógicos “and” y “or”

Elemento Neutro

Definición:

Un elemento neutro para un operador $\oplus$ es un e con la propiedad de que para cualquier x siempre se cumple

$$x \oplus e = e \oplus x = x$$

Ejemplos:

- 0 para la suma
- 1 para el producto
- $+\infty$ / $-\infty$ para Min / Max
- True / False para And / Or

Tipos de Consultas

Consulta de prefijos:

Dado un arreglo v de n posiciones, numeradas de 0 a n − 1, y un operador asociativo ⊕, es posible definir para $0 \leq i \leq n$, la consulta P(i), cuyo resultado es

$$P(i) = v_0 \oplus v_1 \oplus \cdots \oplus v_{i-1}$$

Consulta en rangos:

Dado un arreglo v de n posiciones, numeradas de 0 a n − 1, y un operador asociativo ⊕, es posible definir para $0 \leq i \leq j \leq n$, la consulta R(i, j), cuyo resultado es

$$R(i, j) = v_i \oplus v_{i+1} \oplus \cdots \oplus v_{j-1} \oplus v_j$$

Estructuras que veremos

Veremos estructuras que resuelven consultas de prefijos y en rangos. Si la estructura permite modificaciones, se la denomina **Dinámica**, caso contrario **Estática.**

:v	**Prefijos**	**Rangos**
Estático	Tabla Aditiva	Sparse Table
Dinámico	Binary Indexed Tree (BIT)	Segment Tree

“Orden de dificultad subjetivo”:

Tabla Aditiva < Sparse Table < BIT < Segment Tree

Tablas Aditivas

Una técnica para calcular sumas de subarreglos de manera eficiente usando prefijos acumulativos, funciona para cualquier operador que tenga operador inverso (xor).

Concepto Básico:

- Prefix[i] = suma de los elementos del arreglo hasta i
- La suma de un subarreglo [i, j] es prefix[j] - prefix[i-1].

Ejemplo:

- arr = [2, 4, 5, 7, 8]
- prefix = [2, 6, 11, 18, 26]
- Suma rango [1, 3] = prefix[3] - prefix[0] = 18 - 2 = 16.

Tablas Aditivas código

```
int ar[MAXN], pref[MAXN], n;
void build() {
    pref[0] = ar[0];
    for(int i = 1; i < n; i++)
        pref[i] = pref[i - 1] + ar[i];
}
int query(int b, int e) {
    if(b == 0)
        return pref[e];
    return pref[e] - pref[b - 1];
}
```

Tablas aditivas 2D

Para cada elemento en la matriz original, la tabla aditiva 2D tiene las suma de prefijos considerando todos los elementos anteriores en filas y columnas.

Fórmula:

```
su[i][j] = mat[i][j] + su[i-1][j] + su[i][j-1] - su[i-1][j-1]
```

1	2	3
4	5	6
7	8	9

1	3	6
5	12	21
12	27	45

Tablas aditivas 2D construcción código

```
int mat[MAXN][MAXM], su[MAXN][MAXM], n, m;
void build() {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++)
        {
            su[i][j] = mat[i][j];
            if(i > 0) su[i][j] += su[i - 1][j];
            if(j > 0) su[i][j] += su[i][j - 1];
            if(i > 0 && j > 0)
                su[i][j] -= su[i - 1][j - 1];
        }
    }
}
```

Tablas aditivas 2D

La suma de una submatriz definida por las esquinas superiores izquierdas (r1, c1) y las inferiores derechas (r2, c2) es:

```
su[r2][c2] - su[r1-1][c2] - su[r2][c1-1] + su[r1-1][c1-1]
```

- su[r2][c2]: Suma acumulada desde (0,0) hasta (r2,c2).
- su[r1-1][c2]: Resta la suma acumulada desde (0,0) hasta la fila anterior a la submatriz deseada para eliminar filas adicionales.
- su[r2][c1-1]: Resta la suma acumulada desde (0,0) hasta la columna anterior a la submatriz deseada para eliminar columnas adicionales.
- su[r1-1][c1-1]: Añade la suma acumulada desde (0,0) hasta la esquina superior izquierda de la submatriz para compensar la doble resta.

Tablas aditivas 2D código query

```
int query(int r1, int c1, int r2, int c2) {
    int ans = su[r2][c2];
    if(r1 > 0) ans -= su[r1 - 1][c2];
    if(c1 > 0) ans -= su[r2][c1 - 1];
    if(r1 > 0 && c1 > 0) ans += su[r1 - 1][c1 - 1];
    return ans;
}
```

Range Minimum Query (RMQ)

Range Minimum Query (RMQ) es un tipo de consulta que busca encontrar el elemento mínimo en un subarreglo dado.

Dado el arreglo

[2, 4, 3, 1, 6, 7, 8, 9, 1, 7]

el RMQ del rango [2, 6] devolverá 1

ya que 1 es el mínimo en el subarreglo [3, 1, 6, 7, 8].

Range Minimum Query (RMQ)

Si no trabajamos con un operador con inverso, las tablas aditivas no sirven para calcular intervalos arbitrarios.

Por ejemplo si queremos el mínimo de un intervalo en lugar de la suma, ya no podemos “restar” y hacer lo mismo.

Guardar prefijos es poco. Guardemos el acumulado de más intervalos del arreglo.

Guardaremos el acumulado de todos los intervalos del arreglo con **tamaño potencia de 2**

Sparse Table

La Sparse Table es una estructura de datos que realiza consultas con operadores que cumplan con la propiedad de **idempotencia** (por ejemplo RMQ) en tiempo constante O(1) después de una pre computación en O(n log n).
Se construye una tabla st donde st[j][i] almacena el mínimo en el rango de longitud 2^j comenzando desde el índice i.

UwU	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
val	5	6	8	3	9	4	7	2
0	5	6	8	3	9	4	7	2
1	5	6	3	3	4	4	2	
2	3	3	3	3	2			
3	2							

Sparse table código construcción

```
const int tam = 1000010;
const int logTam = 21;
int n;
int ar[tam], table[logTam][tam];
void inispar()
{
    fore(i, 0, n) table[0][i] = ar[i];
    for(int k = 0; (1 << k) < n; k++)
        for(int i = 0; i + (1 << k) < n; i++)
            table[k + 1][i] = min(table[k][i],
                                  table[k][i + (1 << k)]);
}
```

Sparse Table Queries

Idempotencia: propiedad para realizar una acción determinada varias veces y aun así conseguir el mismo resultado que se obtendría si se realiza una sola vez.

Encontramos el valor de j tal que 2^j es el mayor número menor o igual a la longitud del rango [l, r].

```
j = log2(r - l + 1)
```

Usamos la Sparse Table para encontrar el mínimo en estos dos subrangos que cubren el rango [l, r]:

```
min(st[j][l], st[j][r - (1 << j) + 1])
```

Sparse Table Queries

```
query(3, 7)

j = log2(7 - 3 + 1) = log2(5) = 2

quey(3, 7) = min(st[2][3], st[2][7 - 4 + 1]) =

min(st[2][3], st[2][4]) = 2
```

UwU	0	1	2	3	4	5	6	7
val	5	6	8	3	9	4	7	2
0	5	6	8	3	9	4	7	2
1	5	6	3	3	4	4	2	
2	3	3	3	3	2			
3	2							

Sparse Table queries

En C++, en lugar de usar log2(r - l + 1), utilizamos

31 - __builtin_clz(r - l + 1):

__builtin_clz(x): cantidad de ceros a la izquierda en la representación binaria de x.

Restar esto de 31 (para un entero de 32 bits) nos da el índice del bit más significativo de x, equivalente a floor(log2(x)).

Sparse Table Código Queries

```
int query(int b, int e)
{
    int lev = 31 - __builtin_clz(e - b + 1);
    return min(table[lev][b], table[lev][e - (1 << lev) + 1]);
}
```

Sparse Table Observaciones

- Con sparse table se puede calcular consultas en rangos estáticas para cualquier operador que cuente con idempotencia: Max, Min, GCD, OR, etc
- Se puede usar para calcular el LCA (Lowest Common Ancestor) de dos nodos de un árbol en O(1)
- Combinado con suffix array se puede comparar substrings en O(1)

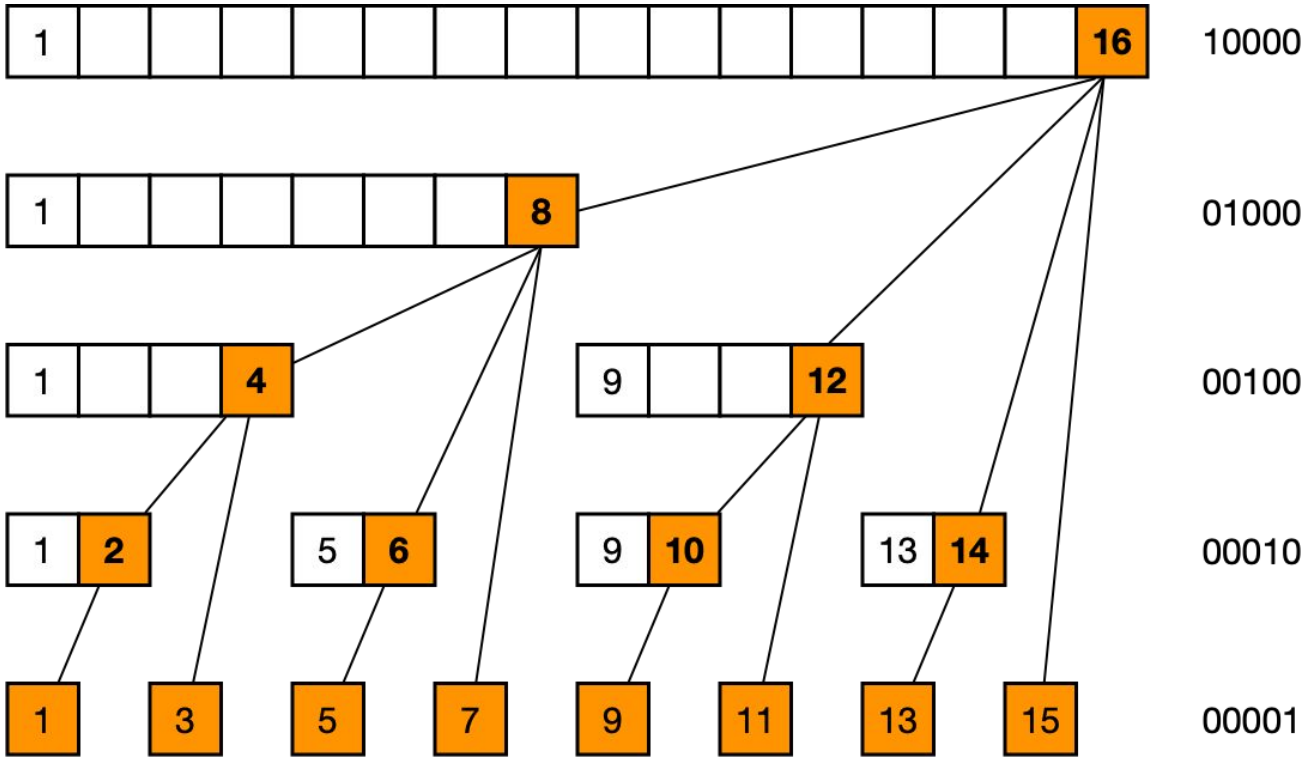
Binary Indexed Tree (BIT)

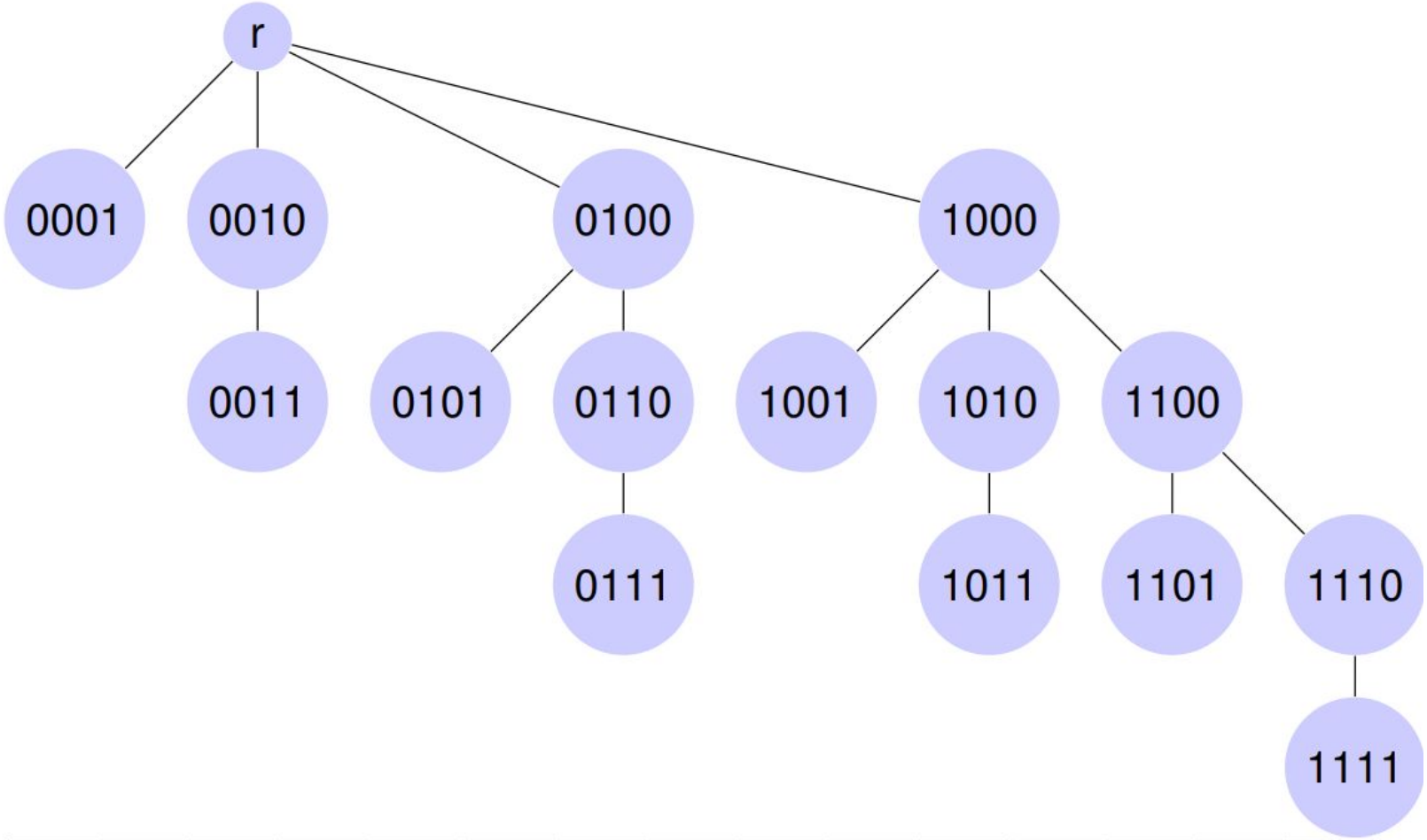
El Binary Indexed Tree (BIT), también conocido como Fenwick Tree, es una estructura de datos que permite realizar actualizaciones y consultas de sumas acumulativas de manera eficiente.

- Responde a consultas y actualizaciones sobre sumas en rangos
- Las consultas devuelven la suma del prefijo
- Para calcular la suma de un rango es igual que la tabla aditiva
- Cada operación cuesta O(log n).
- Fácil de implementar y eficiente en uso de memoria.

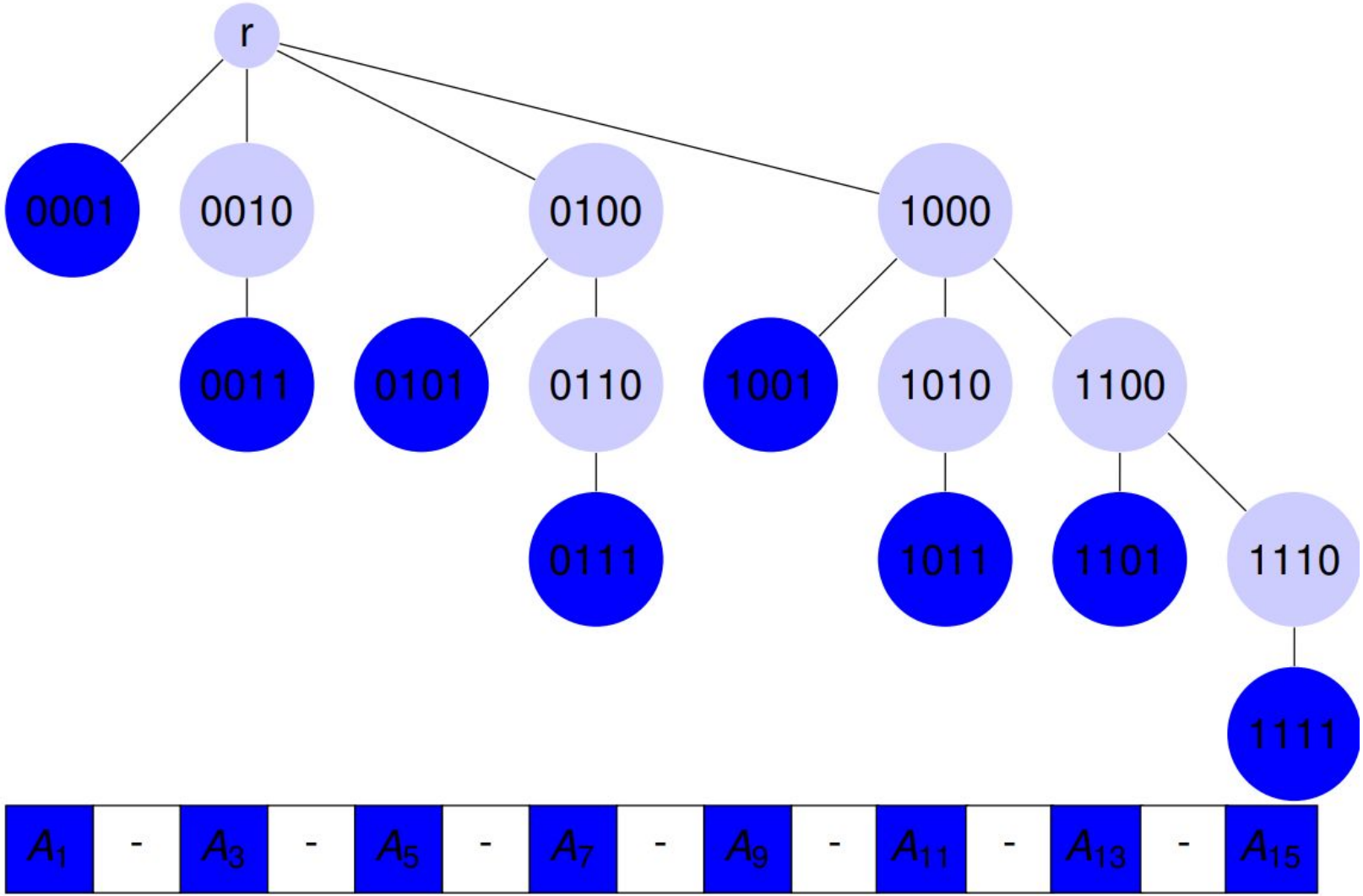
Binary Indexed Tree (BIT)

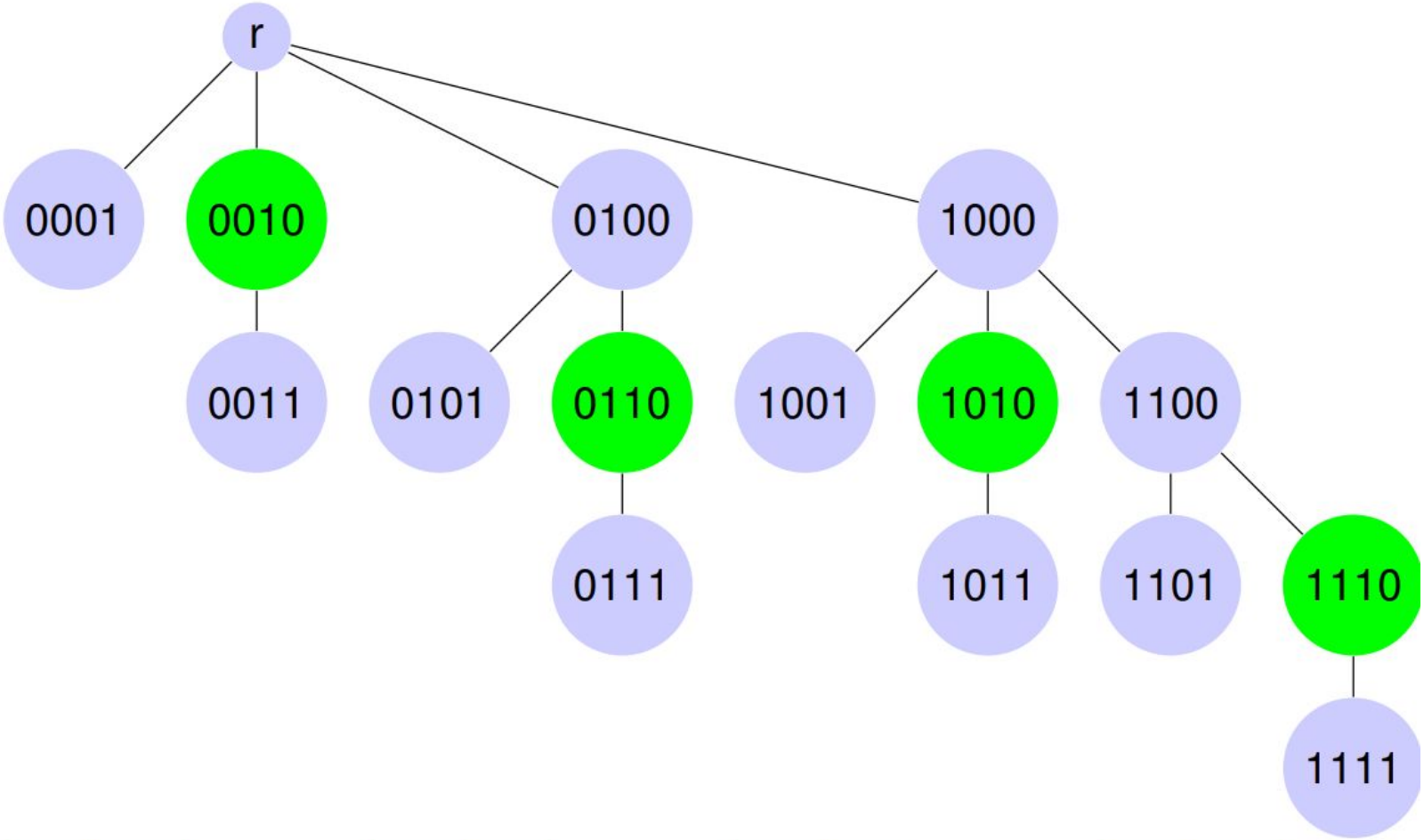
- El BIT se representa como un array donde cada índice almacena información sobre sumas de segmentos del array original.
- Utiliza propiedades de números binarios para determinar los índices afectados durante actualizaciones y consultas.



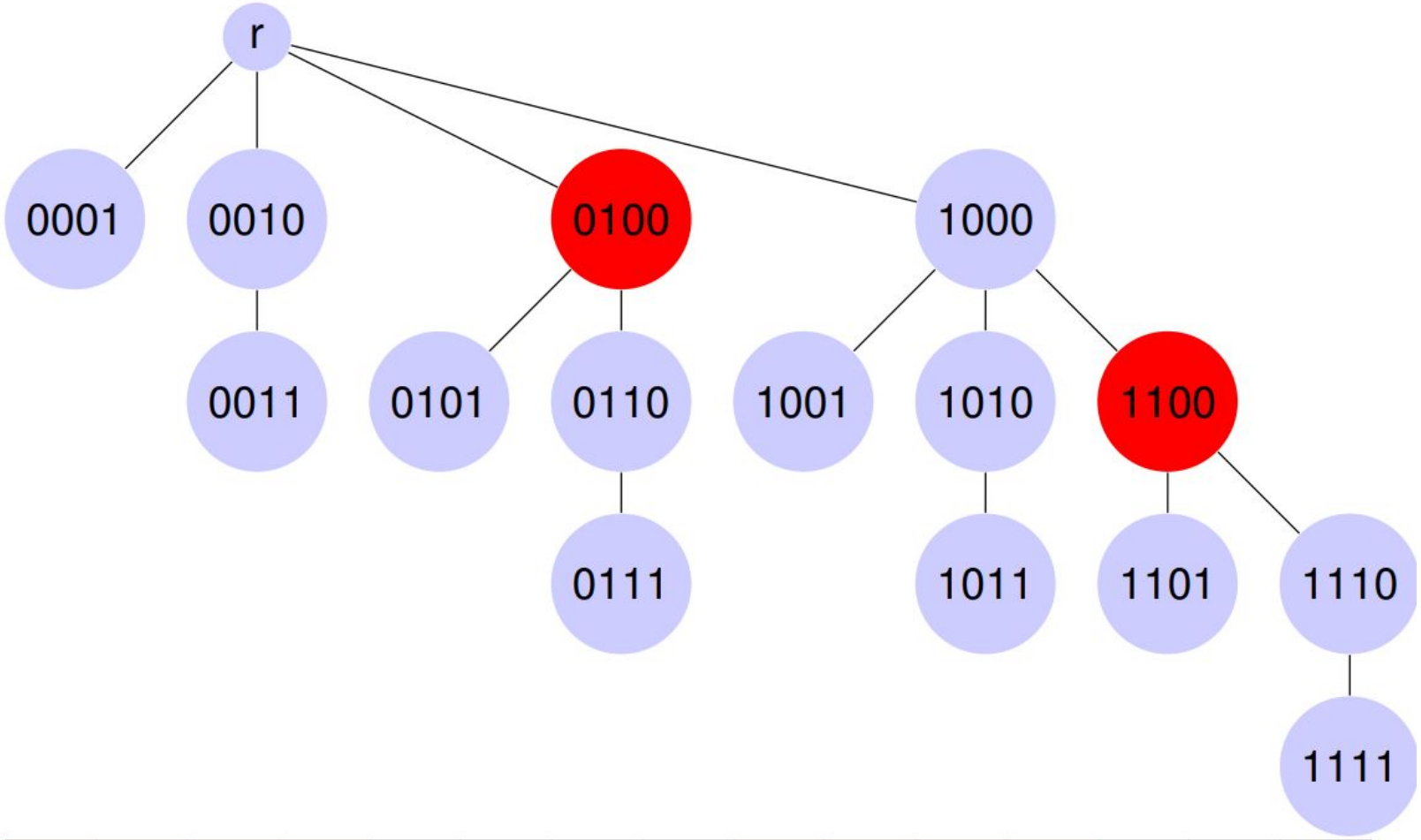

r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

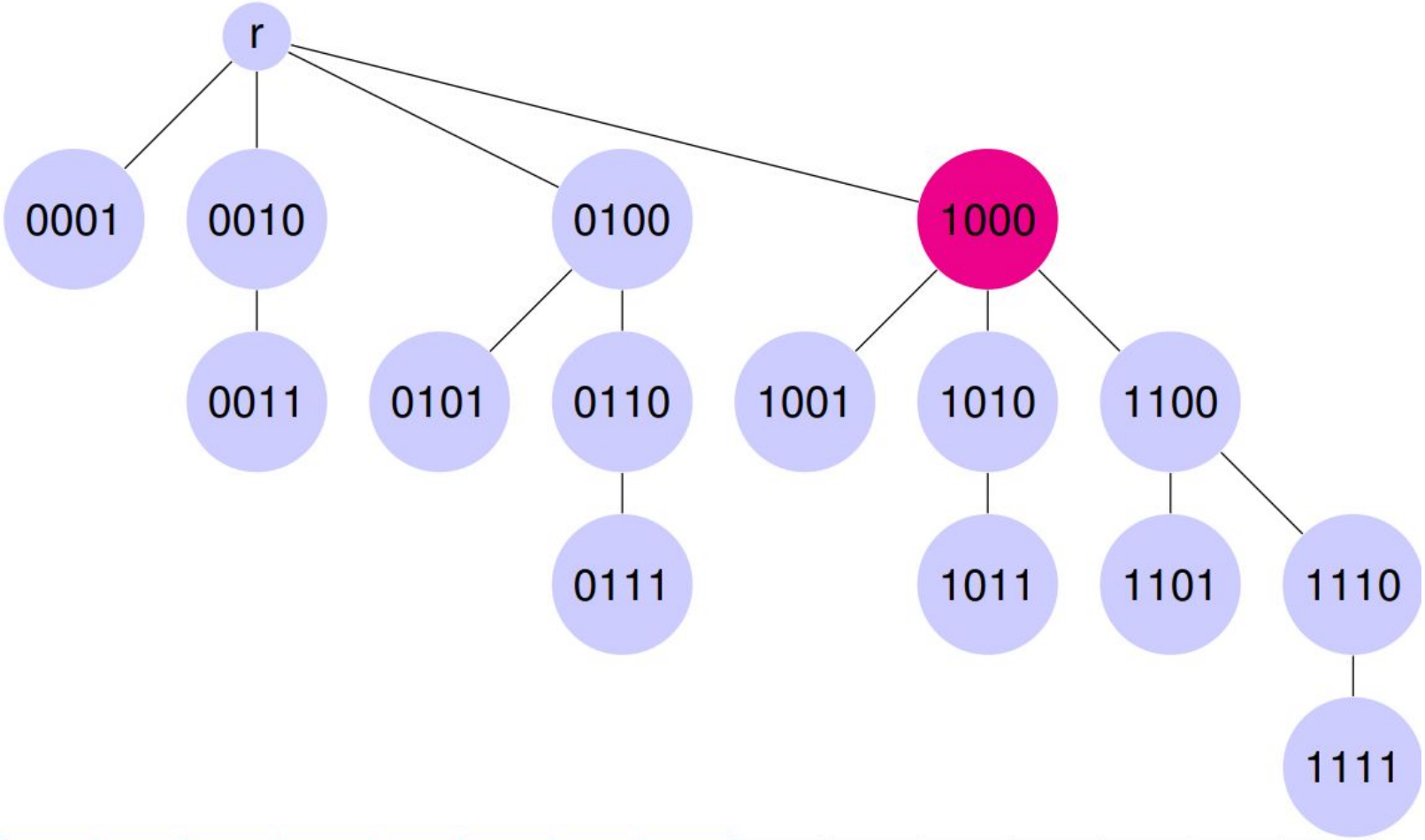

r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111
A_1
-
A_3
-
A_5
-
A_7
-
A_9
-
A_{11}
-
A_{13}
-
A_{15}



A_1	A_2	-	-	A_5	A_6	-	-	A_9	A_{10}	-	-	A_{13}	A_{14}	-



A_1	A_2	A_3	A_4	-	-	-	-	A_9	A_{10}	A_{11}	A_{12}	-	-	-



A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8	-	-	-	-	-	-	-

Binary Indexed Tree (BIT) Actualización

Para actualizar el valor en el índice i del array original, sumamos val a BIT[i] y propagamos la actualización a los índices superiores, sumando al índice el bit menos significativo.

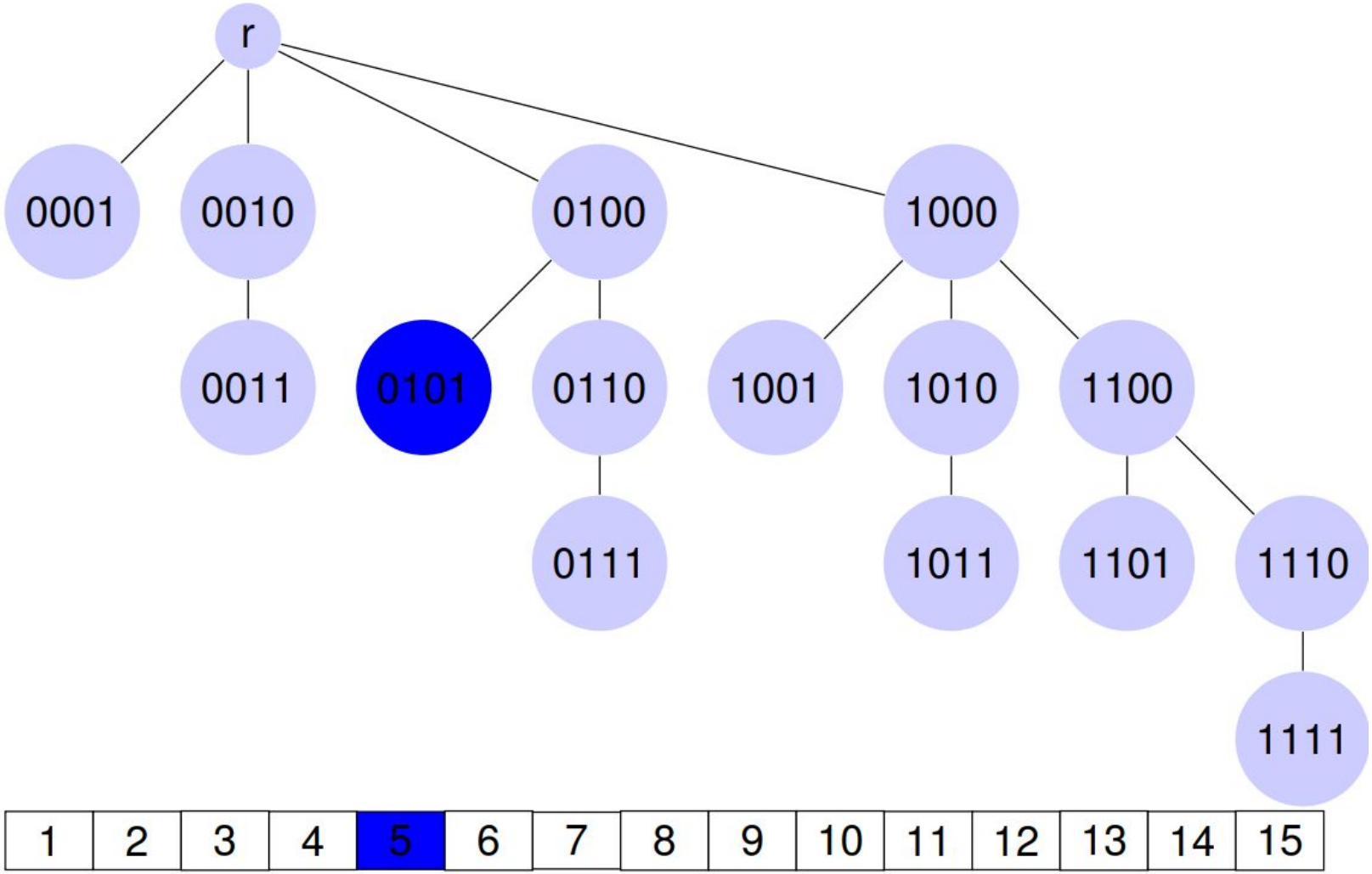
Suponiendo que i = 3, el índice binario es 011.

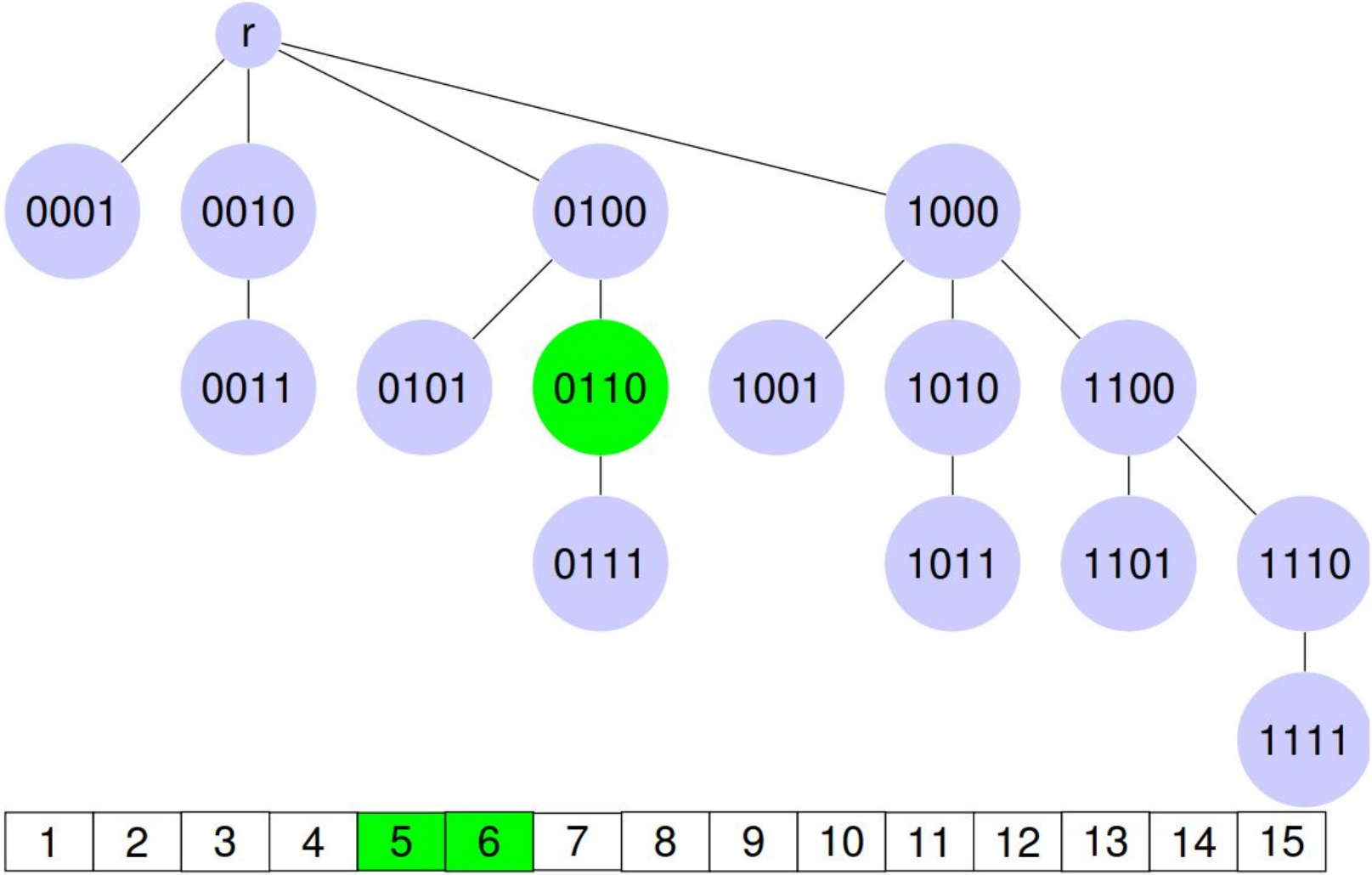
Se actualizan BIT[3] (011), BIT[4] (100), BIT[8] (1000), etc.

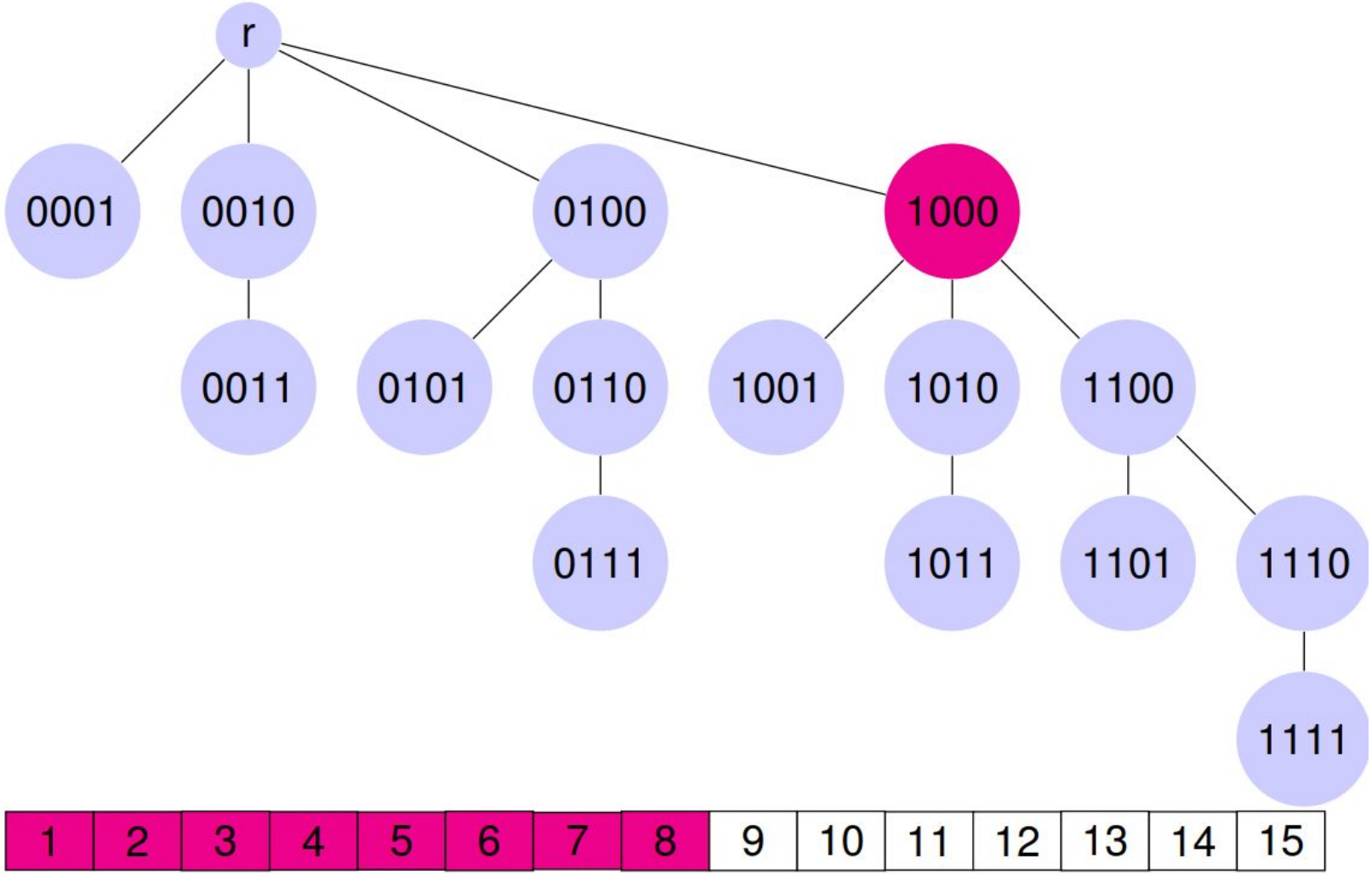
Fórmula de Actualización:

index += index & (-index)

index & (-index) es el bit menos significativo de index


r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Binary Indexed Tree (BIT) Consulta

Para consultar la suma acumulativa hasta el índice i, sumamos los valores en BIT correspondientes a los índices afectados hasta llegar al índice 0, en este caso se resta al índice el bit menos significativo.

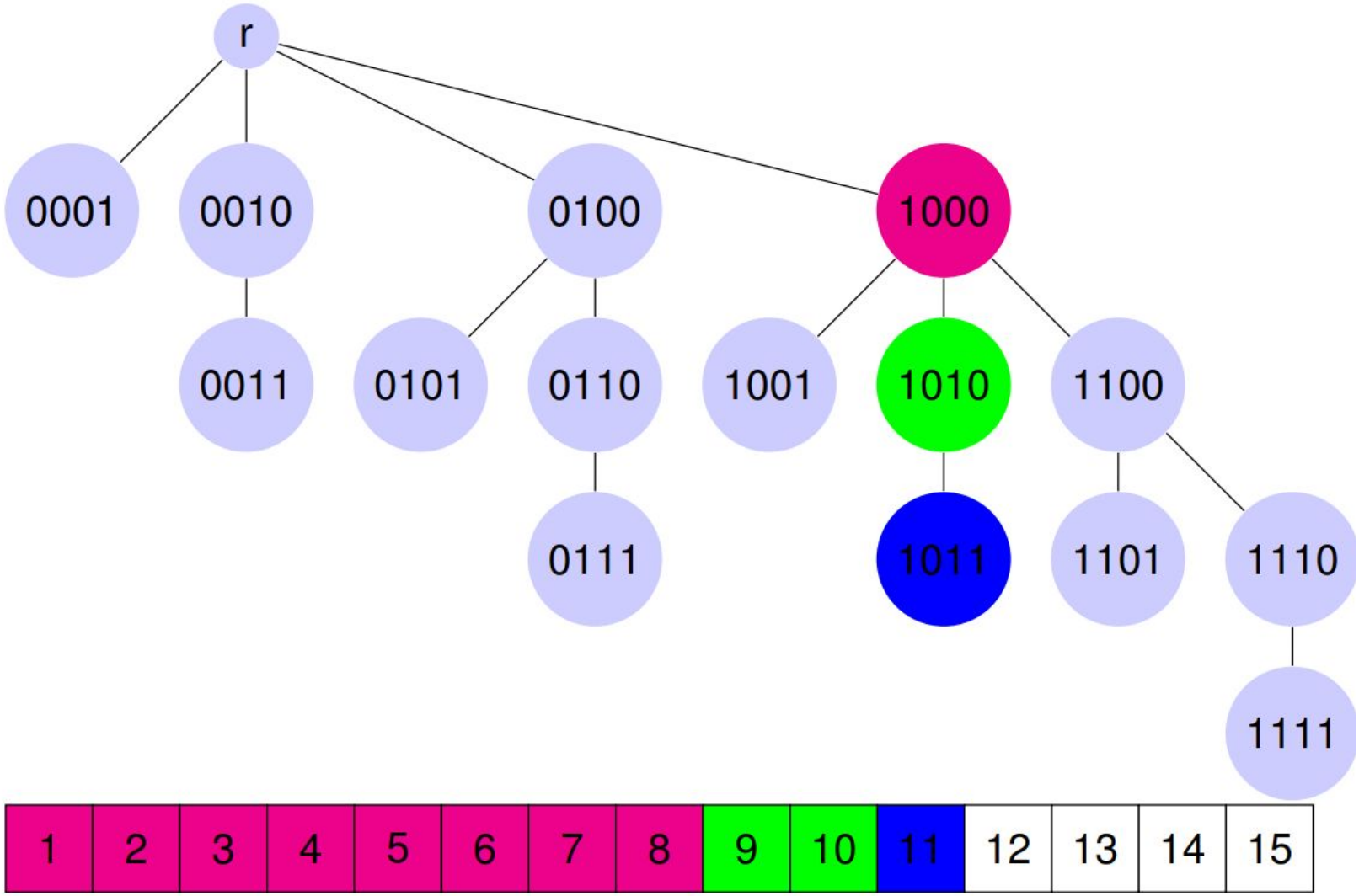
si i = 13, el índice binario es 1101.

Se consultan BIT[13] (1101), BIT[12] (1100), BIT[8] (1000)

Fórmula de Consulta:

index -= index & (-index)

index & (-index) es el bit menos significativo de index


r
0001
0010
0100
1000
0011
0101
0110
1001
1010
1100
0111
1011
1101
1110
1111
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Binary Indexed Tree (BIT) código

```
// usar index 1
int bit[tam];
void update(int pos, int val) {
    while(pos < tam)
        bit[pos] += val, pos += pos & -pos;
}
int query(int pos) {
    int res = 0;
    while(pos > 0)
        res += bit[pos], pos -= pos & -pos;
    return res;
}
int query_rango(int l, int r) {
    return query(r) - query(l-1);
}
```

Binary Indexed Tree 2D (BIT) código

```
// usar index 1
int BIT[tam][tam], n, m;
void update(int row, int col, int val) {
    for (int i = row; i <= n; i += (i & -i)) {
        for (int j = col; j <= m; j += (j & -j))
            BIT[i][j] += val;
    }
}
int query(int row, int col) {
    for (int i = row; i > 0; i -= (i & -i)) {
        for (int j = col; j > 0; j -= (j & -j))
            res += BIT[i][j];
    }
    return res;
}
```

BIT ENLÁCES ÚTILES

https://www.youtube.com/watch?v=CWDQJGaN1gY&ab_channel=TusharRoy-CodingMadeSimple

https://www.youtube.com/watch?v=kPaJfAUwViY&ab_channel=AlgorithmsLive%21

https://cp-algorithms.com/data_structures/fenwick.html

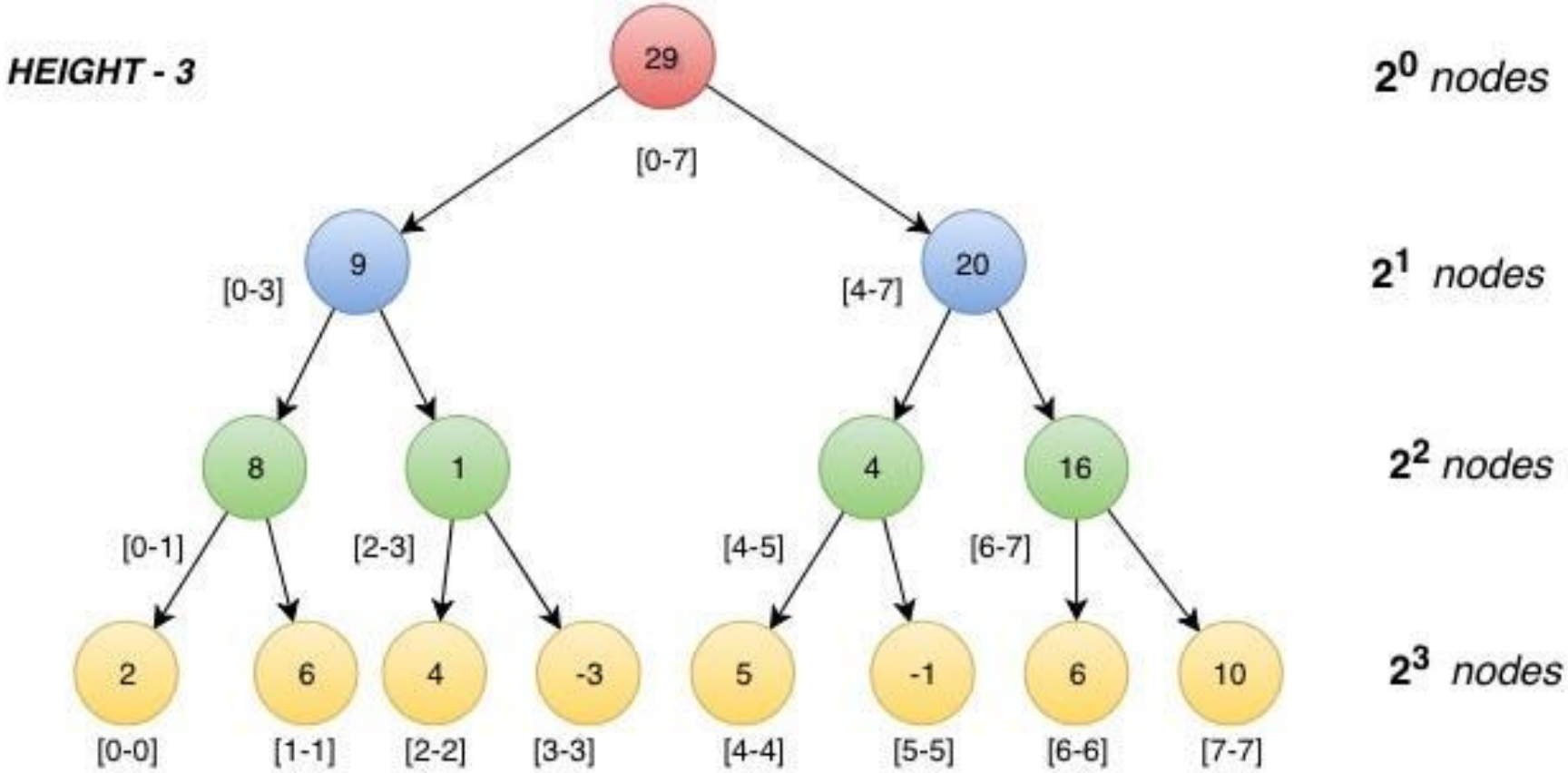
https://usaco.guide/gold/PURS?lang=cpp

Segment Tree

Un Segment Tree es una estructura de datos que permite realizar consultas y actualizaciones en intervalos de manera eficiente.

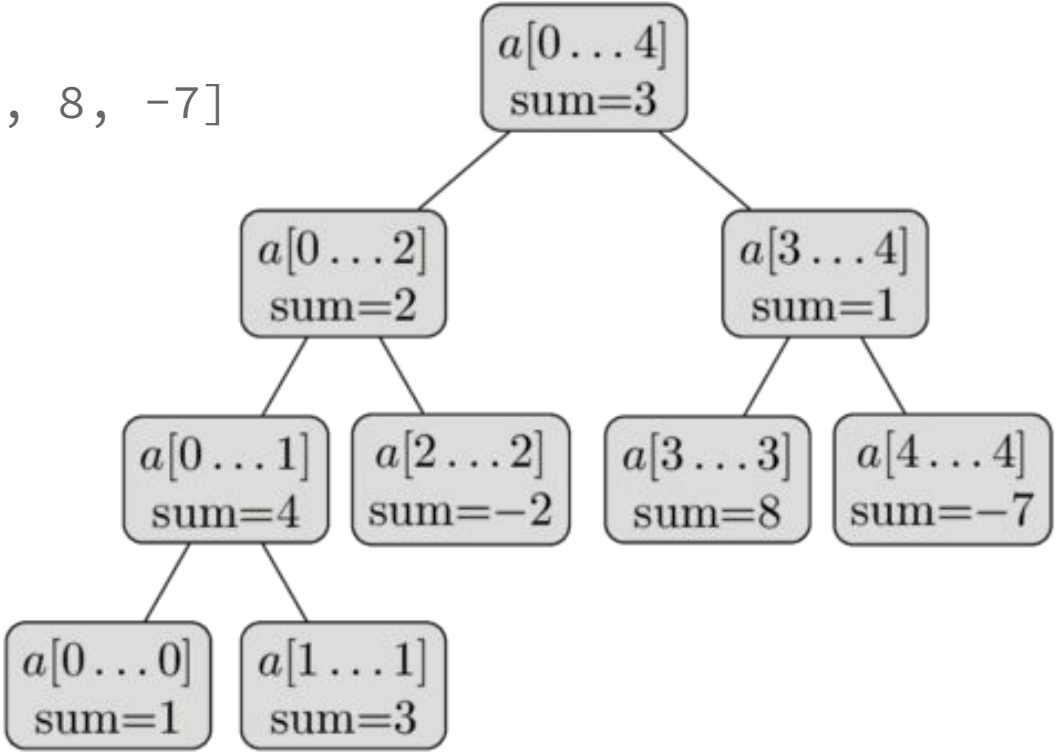
- Es un árbol binario **completo** donde cada nodo representa un intervalo del arreglo.
- La raíz del árbol representa todo el arreglo, y cada hoja representa un solo elemento del array.
- Consultas y actualizaciones en O(log n).
- Flexible para diversos tipos de consultas y actualizaciones.
- Es bastante poderoso

Segment Tree



Segment Tree Representación

```
ar = [1, 3, -2, 8, -7]
```



Segment Tree construcción

- Mantener un arreglo t de tamaño suficiente para almacenar todos los nodos del Segment Tree.
- Generalmente, se necesita un arreglo de tamaño 4 * n donde n es el tamaño del arreglo original.
- Se puede construir el Segment Tree de manera recursiva, comenzando desde la raíz y dividiendo el array en dos mitades en cada paso.
- La raíz representa al arreglo completo
- El hijo izquierdo de la raíz representa al intervalo [0, mid]
- El hijo derecho de la raíz representa al intervalo [mid + 1, n - 1]
- y así sucesivamente hasta las hojas que representan intervalos de tamaño 1, es decir elementos del arreglo.

Segment Tree construcción código

```
struct node {
    int val;
};
node join(node a, node b) {
    node res;
    res.val = a.val + b.val;
    // res.val = min(a.val, b.val);
    // res.val = max(a.val, b.val);
    // res.val = __gcd(a.val, b.val);
    return res;
}
```

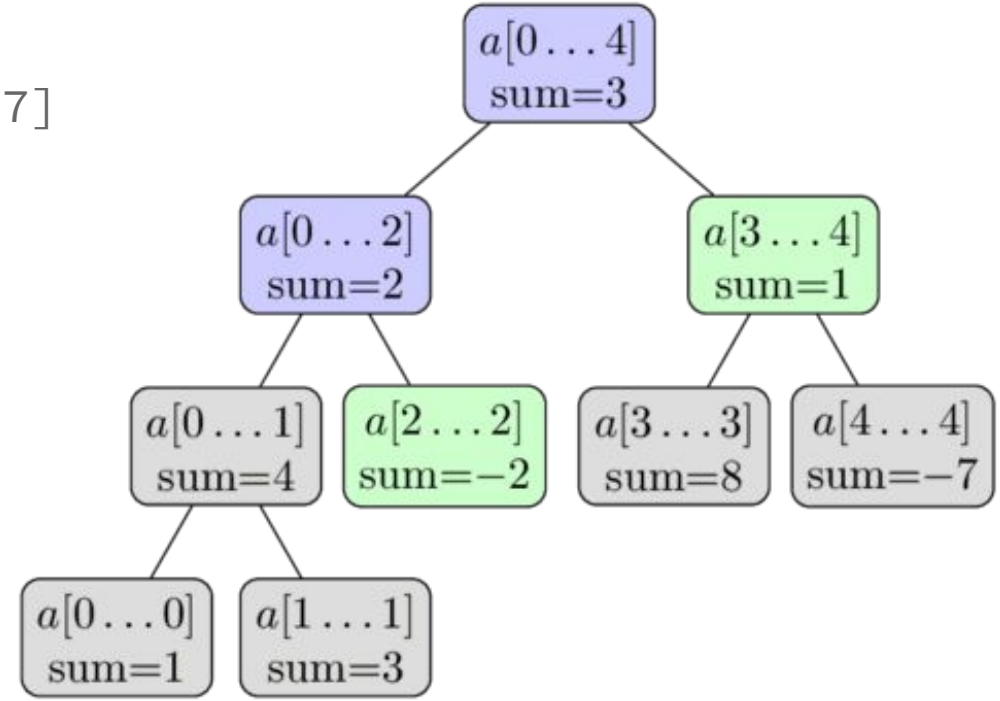
Segment Tree construcción código

```
int ar[tam];
node t[4 * tam];
void init(int b, int e, int node) {
    if(b == e) {
        t[node].val = ar[b];
        return;
    }
    int mid = (b + e) / 2, l = node * 2 + 1, r = l + 1;
    init(b, mid, l);
    init(mid + 1, e, r);
    t[node] = join(t[l], t[r]);
}
```

Segment tree queries

```
ar = [1, 3, -2, 8, -7]

Query entre 2 y 4
```



Segment Tree Queries

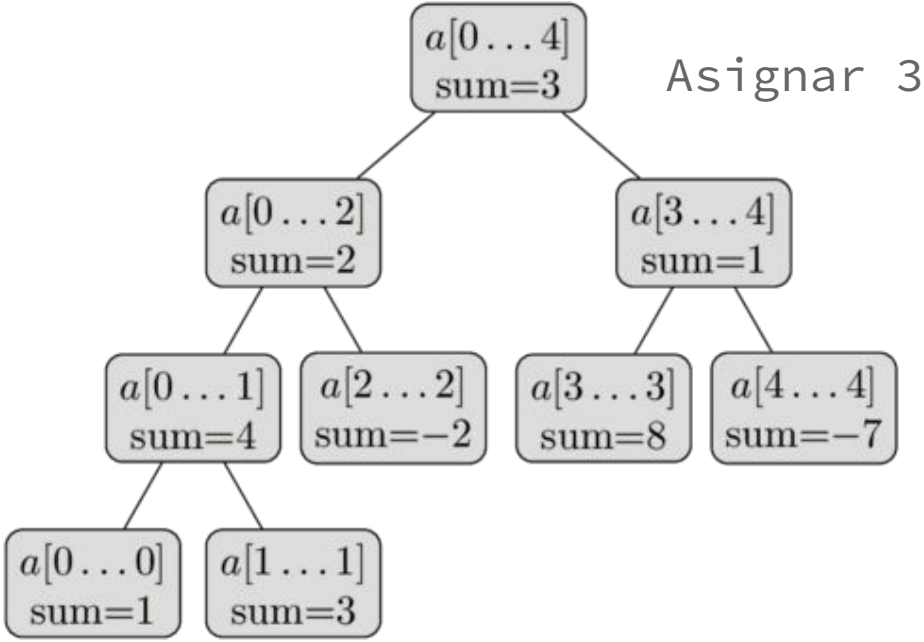
Para hacer una consulta lo que haremos es ver los nodos empezando por la raíz. Al ver un nodo tenemos 2 posibilidades:

- Que el rango que el nodo representa esté totalmente contenido en la consulta, caso en el que devuelve su valor.
- Que el rango esté parcialmente contenido por la consulta, caso en el que preguntara a sus hijos y aplicará la operación al resultado de estos.

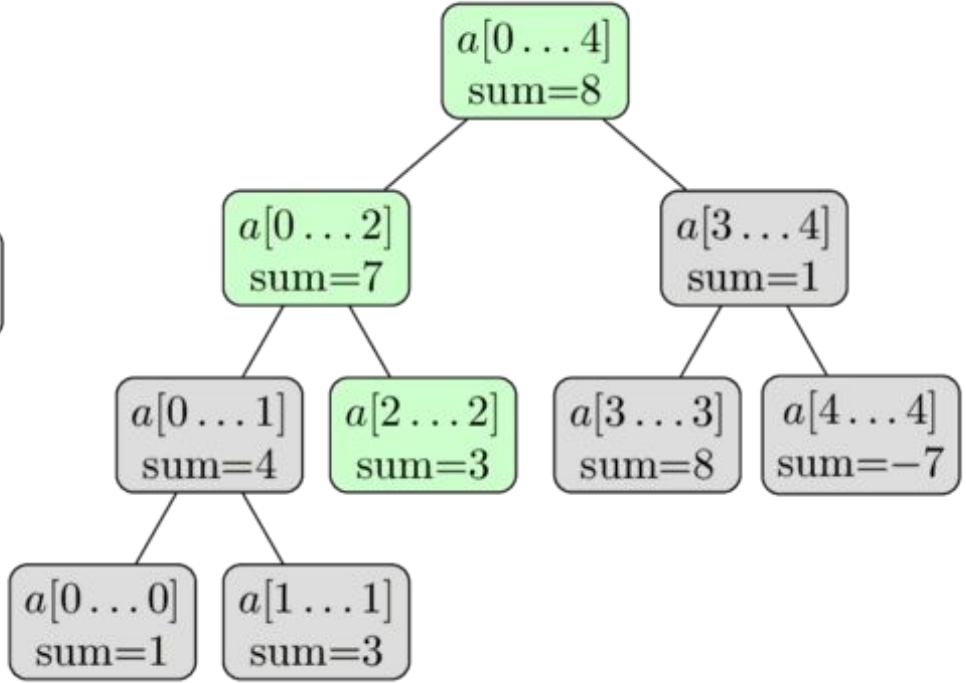
Segment Tree Queries código

```
node query(int b, int e, int node, int i, int j) {
    if(b >= i && e <= j)
        return t[node];
    int mid = (b + e) / 2, l = node * 2 + 1, r = l + 1;
    if(mid < i)
        return query(mid + 1, e, r, i, j);
    if(mid >= j)
        return query(b, mid, l, i, j);
    return join(query(b, mid, l, i, j),
                query(mid + 1, e, r, i, j));
}
```

Segment tree update



Asignar 3 en la posición 2



Segment tree update

Para hacer la actualización o cambio:

- Iremos a la hoja que representa el nodo que queremos actualizar y cambiaremos su valor.
- Al cambiar este elemento, solo cambian los valores de los rangos que lo contienen. Es decir, solo cambian los valores de los ancestros del nodo en nuestro árbol.
- Actualizamos los ancestros empezando por su padre, luego el padre de éste, y así hasta llegar a la raíz.

Segment tree update código

```
void update(int b, int e, int node, int pos, int val) {
    if(b == e) {
        t[node].val = val;
        return;
    }
    int mid = (b + e) / 2, l = node * 2 + 1, r = l + 1;
    if(mid < pos)
        update(mid + 1, e, r, pos, val);
    else
        update(b, mid, l, pos, val);
    t[node] = join(t[l], t[r]);
}
```

Segment tree enlaces útiles

https://www.youtube.com/watch?v=ZBHKZF5w4YU&ab_channel=TusharRoy-CodingMadeSimple

https://www.youtube.com/watch?v=Tr-xEGoByFQ&ab_channel=AlgorithmsLive%21

https://cp-algorithms.com/data_structures/segment_tree.html

https://usaco.guide/gold/PURS?lang=cpp

https://usaco.guide/plat/segtree-ext?lang=cpp

Problemas (este hubiera sido el contest :'v)

https://cses.fi/problemset/task/1646

https://cses.fi/problemset/task/1647

https://cses.fi/problemset/task/1648

https://cses.fi/problemset/task/1649

https://cses.fi/problemset/task/1650

https://cses.fi/problemset/task/1651

https://cses.fi/problemset/task/1652

https://codeforces.com/contest/339/problem/D

https://www.spoj.com/problems/GSS3/

https://www.spoj.com/problems/GSS4/